SSQ-LR: Sub-FP16 Recurrent State Quantization for Hybrid Reasoners

Anonymous authors
Paper under double-blind review

Abstract

This is a living paper shell for a preregistered Mac-first experiment. SSQ-LR tests whether recurrent SSM state in hybrid reasoning models can be quantized below FP16 while preserving quality. No positive result or native systems claim is made yet; current held-out S2 scouts fail to produce a recipe that clears both quality and byte-accounting gates.

1 Current Gate and Claim Boundary

The branch must pass state-distribution heterogeneity, state-only quantization sensitivity, and cross-model transfer gates before any GPU validation. S1 must first show real state heterogeneity on a live hybrid model; S2 then freezes one state-only quantization recipe; S3 tests that recipe without per-model retuning. Failing any gate kills the branch before GPU work. The current real trace packet builder/checker covers S1. S2 and S3 now have executable follow-up contracts in `experimental/shared/followup_gate_contracts.py`: they require frozen recipe IDs, byte accounting, BF16 no-op controls, paired accuracy and continuation-NLL quality bounds, verbosity/length-drift readouts before revival, frozen recipe/source hashes, and no-retuning transfer rows. Resource-limited S2 scouts are not current evidence for promotion. SSQ-LR makes no quality, byte-savings, transfer, GPU, latency, or throughput claim until non-resource-limited S1–S3 packets clear.

2 Mac Artifact Status

A deterministic synthetic S1 packet now validates the real-schema checker path before real hybrid-model state dumps. It emits 288 rows covering 12 prompts, 6 recurrent layers, and the four preregistered buckets, producing `SCHEMA_REHEARSAL_NOT_PROMOTABLE_SYNTHETIC_SSQ_LR_S1`. This is synthetic-only: it is not model evidence, not a quality result, and not GPU evidence. The held-out S1b packet is the strongest positive evidence so far: it repeats the frozen selected layers `0`, `12`, and `30` on 12 held-out prompts with selected ratio 2.459 and lower bound 1.861. Uniform state recipes do not yet convert that heterogeneity into a method. MXFP4 preserves short BF16-argmax fidelity but reaches only 3.765–3.938× state-memory reduction after scale bytes. INT3 reaches 4.923–5.224× after scale bytes, but both 12-prompt held-out scouts fail because at least one prompt loses argmax fidelity. The 4-prompt INT3 pass is therefore treated as resource-limited and non-promoting. The newest resource-limited mixed-block S2b scout selects `mixed_int3_mxfp4_low_error_25pct`: it reaches 4.192× counted state-memory reduction on the 12-prompt held-out replay with zero BF16-argmax delta and 0.03956 selected NLL-delta CI high. This short-window pass is still simulated replay and cannot promote to GPU. The same recipe then fails a longer-window replay (`--max-input-tokens 24 --prefix-tokens 8`): it preserves the 4.192× byte readout but has selected accuracy CI high 0.0667 and selected NLL-delta CI high 0.0764. The mixed recipe is therefore stopped before GPU validation.

3 Next Evidence

The next admissible action is not GPU validation. SSQ-LR should either run a single-layer S2 localization scout under the same longer-window replay or stop the branch as diagnostic S1b evidence. Otherwise the S1b/S2b result is diagnostic evidence, not a positive method.

4 Admissible Real Packet

Promotable packets must pin model/tokenizer revisions, prompt and trace-plan hashes, the shared architecture-map hash, exact command/device/dtype metadata, and saved recurrent-state tensor hashes. S1 rows must cover all four registered position buckets, including `prefill_end`, for every prompt/layer pair and at least 12 prompt IDs unless explicitly marked resource-limited. The checker recomputes row metrics and gate aggregates from raw tensors, rejects stale summaries, and requires `RESOURCE_LIMITED_NOT_PROMOTABLE` decisions for any subset packet. Full schema details live in the README and runbook.

Required baseline	Purpose
BF16 no-op replay	Catch hook/serialization bugs.
FP16 stochastic-rounding SSM cache	Deployment baseline from Nemotron-style hybrid serving.
INT16 block-scaled SSM cache	Higher-precision recurrent-cache control.
INT8 state-only	Standard low-bit state control.
FP8-style state-only	Checks whether sub-FP16 state can match the likely deployed fallback.
MXFP4 state-only	Candidate sub-FP16 recipe.
INT3 state-only	Tests whether sub-4-bit state can clear scale-byte overhead.
Random same-L2 noise	Separate structure from noise magnitude.
Scale shuffle	Test scale assignment sensitivity.
Byte accounting	Include scale/metadata overhead.

Table 1: Controls required before SSQ-LR can claim a state-quantization recipe.

5 Reviewer Packet

The reviewer-facing claim boundary is tracked in `experimental/ssq_lr/paper/reviewer_pack.md`. It records that SSQ-LR is not camera-ready as a method paper until real S1–S3 evidence exists. The newest novelty baseline is Nemotron 3 Super's recurrent-cache recipe: NVIDIA documentation treats Mamba SSM cache quantization as a separate deployment challenge and selects FP16 with stochastic rounding for the SSM cache, with INT16/block-scaled variants discussed as controls. SSQ-LR is therefore not novel for noticing SSM cache precision; it must beat this baseline with a sub-FP16 recipe and paired quality evidence. The source memo is `../../../references/754_ssq_lr_nemotron_state_cache_refs_20260507.md`.

6 Claim Boundary

SSQ-LR is only about recurrent SSM-state precision in hybrid reasoners. It does not claim weight quantization, activation quantization, KV-cache quantization, native serving speed, or memory savings until a real state recipe and later GPU validation pass their preregistered gates.